

TijaraFlow

Documentation technique

Guide administrateurs & développeurs

Version 1.0

Édition 2026 · Architecture, exploitation, extension

Table des matières

Table des matières.....	2
1. Introduction.....	4
1.1 Périmètre.....	4
1.2 Conventions.....	4
2. Vue d'ensemble de l'architecture.....	4
2.1 Composants.....	4
2.2 Flux principal.....	5
3. Stack technique.....	5
4. Structure du dépôt.....	6
5. Environnements et configuration.....	6
6. Modèle de données.....	7
6.1 Tables principales.....	7
7. Fonctions PostgreSQL.....	9
8. Authentification et autorisation.....	9
8.1 Sessions.....	9
8.2 Middleware et routes publiques.....	9
8.3 Réinitialisation de mot de passe.....	10
8.4 Multi-tenancy.....	10
9. Forfaits et entitlements.....	10
9.1 Points d'application.....	10
10. Architecture de synchronisation.....	11
10.1 Étapes du handler.....	11
10.2 Ajouter une plateforme.....	11
11. Intégrations boutiques.....	11
12. Facturation Stripe.....	12
12.1 Création d'abonnement.....	12
12.2 Webhook Stripe.....	12
13. E-mails (Resend).....	12

14. Tâches planifiées (crons).....	12
15. Observabilité.....	13
16. Rate limiting.....	13
17. Déploiement.....	13
17.1 Pipeline.....	13
17.2 Configuration.....	13
17.3 Migrations base de données.....	14
18. Développement local.....	14
19. Sécurité.....	14
20. Runbook d'incident.....	14
21. Étendre la plateforme.....	15
21.1 Ajouter un forfait.....	15
21.2 Ajouter un endpoint API.....	15
21.3 Ajouter une automatisation.....	15
22. Annexe — Référence des endpoints.....	16

1. Introduction

Ce document s'adresse aux administrateurs et développeurs qui déploient, exploitent ou étendent TijaraFlow. Il décrit l'architecture, le modèle de données, les mécanismes de sécurité, les intégrations, le déploiement et les procédures d'exploitation.

1.1 Périmètre

- Architecture applicative et choix techniques.
- Modèle de données PostgreSQL et fonctions serveur.
- Authentification, autorisation et multi-tenancy.
- Synchronisation des boutiques, facturation Stripe, e-mails, tâches planifiées.
- Déploiement, observabilité et runbook d'incident.
- Procédures d'extension (nouvel adaptateur, nouveau forfait, nouvel endpoint).

1.2 Conventions

Les chemins de fichiers sont relatifs à la racine du dépôt. Les blocs de code sont indicatifs. Les noms de variables d'environnement sont sensibles à la casse.

2. Vue d'ensemble de l'architecture

TijaraFlow est une application Next.js 16 (App Router) déployée sur Vercel, adossée à Supabase (PostgreSQL + Auth) pour les données et l'authentification, à Stripe pour les abonnements, et à Resend pour les e-mails. Les boutiques e-commerce communiquent via des webhooks traités par une couche d'adaptateurs.

2.1 Composants

Composant	Rôle
Next.js 16 (App Router)	Front-end (RSC) + back-end (Route Handlers, Server Actions).
Supabase Postgres	Base de données, RLS, fonctions SQL.
Supabase Auth (SSR)	Authentification par cookies, sessions côté serveur.
Stripe	Abonnements (Checkout + webhooks).

Composant	Rôle
Resend	Envoi des e-mails transactionnels.
Vercel	Hébergement, déploiement continu, cron jobs.
Adaptateurs sync	WooCommerce, Shopify (webhooks entrants).

2.2 Flux principal

1. Une boutique émet un webhook vers `/api/webhooks/{plateforme}`.
2. Le handler authentifie le tenant (clé API), applique le rate limiting, vérifie la signature.
3. L'adaptateur normalise le payload en un modèle commun.
4. Le cœur (`lib/sync/core`) applique la mutation (produit, commande, client, stock).
5. L'événement est journalisé (`sync_events`) ; les automatisations éligibles se déclenchent.

3. Stack technique

Domaine	Technologie
Framework	Next.js 16, React 19, TypeScript strict
Styles	Tailwind CSS v4, shadcn/ui (Radix)
Données	PostgreSQL (Supabase), RLS activée
Auth	Supabase Auth via <code>@supabase/ssr</code>
Paielements	Stripe (mode abonnement)
E-mails	Resend
Graphiques	Recharts
Hébergement	Vercel (cron, env, déploiement Git)

4. Structure du dépôt

app/	Routes (App Router)
(marketing) /, /tarifs, /fonctionnalites, /contact	
login, register, reset, reset/update	
dashboard/	Espace connecté (stocks, orders, ...)
api/v1/	API publique (products, orders, ...)
api/webhooks/	woocommerce, shopify, stripe
api/billing/checkout	Création de session Checkout
api/cron/	daily, weekly
api/export/[entity]	Exports CSV
api/health	Sonde de santé
lib/	
supabase/	clients server/client + database.types
sync/	types, core, webhook-handler, adapters/
entitlements.ts	plans, limites, gating
plans.ts	définition des forfaits
email/	resend, send
rate-limit.ts	anti-abus
observability.ts	journal d'erreurs
components/	ui (primitives), shared, app, dashboard
middleware.ts	garde d'authentification + routes publiques

5. Environnements et configuration

Les variables d'environnement sont définies dans Vercel (Production) et, en local, dans `.env.local`. Ne jamais committer de secret.

Variable	Rôle	Secret
NEXT_PUBLIC_SUPABASE_URL	URL du projet Supabase	Non
NEXT_PUBLIC_SUPABASE_ANON_KEY	Clé publique Supabase	Non
SUPABASE_SERVICE_ROLE_KEY	Clé service (bypass RLS, serveur uniquement)	Oui

Variable	Rôle	Secret
STRIPE_SECRET_KEY	Clé secrète Stripe	Oui
STRIPE_WEBHOOK_SECRET	Secret de signature du webhook Stripe	Oui
STRIPE_PRICE_PRO	ID de prix du forfait Pro	Non
STRIPE_PRICE_BUSINESS	ID de prix du forfait Business	Non
RESEND_API_KEY	Clé API Resend	Oui
RESEND_FROM_EMAIL	Expéditeur des e-mails (défaut onboarding@resend.dev)	Non
CRON_SECRET	Jeton d'autorisation des crons	Oui
NEXT_PUBLIC_APP_URL	URL publique de l'app	Non

Règle de sécurité

La clé SUPABASE_SERVICE_ROLE_KEY contourne la RLS. Elle ne doit être utilisée que côté serveur (route handlers, crons, webhooks) et jamais exposée au navigateur.

6. Modèle de données

Toutes les tables métier portent une colonne `user_id` et sont protégées par RLS : chaque utilisateur n'accède qu'à ses propres lignes. Les écritures système (synchro, webhooks, crons) passent par la clé service.

6.1 Tables principales

Table	Contenu
products	Catalogue : name, sku, price, cost, stock_quantity, low_stock_threshold, external_id/source, woo_id.
customers	Clients : full_name, email, phone, adresse.

Table	Contenu
orders	Commandes : status, total_amount, customer_id, external_id/source.
order_items	Lignes de commande : product_id, quantity, unit_price.
invoices	Factures : invoice_number, status, amount, due_date, paid_at.
api_keys	Clés API : key_prefix, key_hash (sha256), last_used_at.
user_settings	Réglages : notify_email, auto_invoice, stock_alerts, overdue_reminders, weekly_report, wc_webhook_secret.
user_subscriptions	Abonnement : plan, status, stripe_customer_id, stripe_subscription_id, current_period_end.
stock_movements	Registre de stock : delta, reason, balance_after, reference.
sync_events	Journal de synchro : source, action, status, detail.
error_logs	Journal d'erreurs : source, context, message, details.
rate_limits	Compteurs anti-abus : bucket, count, window_start.
demo_requests	Demandes de démo (formulaire landing).
contact_messages	Messages de contact (page Contact).

RLS

Les tables exposées à l'utilisateur ont des politiques « select/insert own » basées sur `auth.uid() = user_id`. `sync_events` et `stock_movements` sont en lecture seule côté utilisateur ; les insertions se font via la clé service ou des fonctions `SECURITY DEFINER`.

7. Fonctions PostgreSQL

Fonction	Signature et rôle
decrement_stock	(p_product_id, p_quantity, p_reason='sale', p_reference) : décrémente le stock et journalise un mouvement.
increment_stock	(p_product_id, p_quantity, p_reason='restock', p_reference) : incrémente et journalise.
generate_invoice_number	() → texte : numéro de facture séquentiel.
check_rate_limit	(p_bucket, p_limit, p_window_seconds) → booléen : fenêtre fixe atomique, fail-open.
forecast_stock	(p_user_id, p_history_days=30, p_lead_time_days=7, p_cover_days=30) → table : vitesse, couverture, réappro.
reports_overview	(p_user_id, p_days=30) → jsonb : CA, top produits, valeur stock, clients.

Les fonctions de stock et de rapports sont SECURITY DEFINER et filtrent explicitement par user_id.

8. Authentification et autorisation

8.1 Sessions

L'authentification utilise Supabase Auth via @supabase/ssr. Le client serveur lit/écrit les cookies de session ; le middleware rafraîchit la session et protège les routes.

8.2 Middleware et routes publiques

middleware.ts redirige tout utilisateur non authentifié vers /login, sauf pour les routes publiques : /, /tarifs, /fonctionnalites, /contact, /login, /register, /reset, /auth, /payment, /api/webhooks, /api/v1, /api/health, /api/demo-request, /api/contact.

8.3 Réinitialisation de mot de passe

6. `/reset` appelle `resetPasswordForEmail` avec `redirectTo = /auth/callback?next=/reset/update`.
7. `/auth/callback` échange le code et redirige vers la cible interne.
8. `/reset/update` appelle `updateUser({ password })`.

8.4 Multi-tenancy

L'isolation repose sur `user_id` + RLS. Les opérations système utilisent la clé service et filtrent manuellement par `user_id`.

9. Forfaits et entitlements

`lib/plans.ts` définit trois forfaits (Starter, Pro, Business) avec leurs limites (products, orders, stores) et leurs fonctionnalités (automations, ai, api). `lib/entitlements.ts` applique ces droits.

Helper	Rôle
<code>getUserPlan(supabase, userId)</code>	Renvoie le plan effectif (abonnement actif sinon Starter).
<code>canCreate(supabase, userId, resource, qty)</code>	Vérifie la limite produits/commandes avant création.
<code>canUseStore(supabase, userId, source)</code>	Vérifie la limite de boutiques (multi-store).
<code>hasAutomations / usersWithAutomations</code>	Gate l'exécution des automatisations (Pro+).

9.1 Points d'application

- Création produit/commande (actions dashboard + API v1).
- Accès API publique (réservé au forfait Business) dans `withApiAuth`.
- Connexion d'une nouvelle boutique dans le handler de webhook.
- Exécution des automatisations (core sync, orders, crons).

10. Architecture de synchronisation

La synchronisation suit un patron d'adaptateurs : un cœur normalisé et un adaptateur par plateforme.

```
lib/sync/types.ts           Modèle normalisé + interface PlatformAdapter
lib/sync/core.ts           Logique métier (upsertProduct, createOrder, ...)
lib/sync/webhook-handler.ts Auth clé API → rate limit → signature → dispatch
lib/sync/adapters/woocommerce.ts
lib/sync/adapters/shopify.ts
lib/sync/sync-log.ts        Journalisation des events (sync_events)
```

10.1 Étapes du handler

9. Authentification du tenant via le paramètre `api_key` (hash sha256 → `user_id`).
10. Rate limiting par tenant (bucket `wh:<userId>`).
11. Vérification de signature HMAC si un secret est configuré.
12. Parsing par l'adaptateur → action normalisée.
13. Gating de boutique (`canUseStore`) pour les nouvelles sources.
14. Dispatch vers le cœur, puis journalisation (succès/erreur).

10.2 Ajouter une plateforme

15. Créer `lib/sync/adapters/<plateforme>.ts` implémentant `PlatformAdapter` (`source`, `verifySignature`, `parse`).
16. Créer la route `app/api/webhooks/<plateforme>/route.ts` appelant `handleSyncWebhook(req, adapter)`.
17. Rendre la route publique dans `middleware.ts` (déjà couvert par `/api/webhooks`).

11. Intégrations boutiques

La configuration côté boutique consiste à pointer un webhook vers TijaraFlow et à activer les événements pertinents.

URL : `https://<app>/api/webhooks/woocommerce?api_key=sf_live_xxx`

WooCommerce : `product.created/updated/deleted`, `order.created/updated`, `customer.created/updated`, `order.deleted`. Shopify : `products/orders/customers create-update-`

delete. Une signature HMAC peut être activée via `user_settings.wc_webhook_secret` (secret partagé du tenant).

12. Facturation Stripe

12.1 Création d'abonnement

`app/api/billing/checkout` crée une session Checkout en mode subscription. L'ID de prix provient des variables `STRIPE_PRICE_PRO` / `STRIPE_PRICE_BUSINESS`. Les métadonnées portent `user_id` et `plan`.

12.2 Webhook Stripe

`app/api/webhooks/stripe` vérifie la signature (`STRIPE_WEBHOOK_SECRET`) et traite : `checkout.session.completed` (activation d'abonnement ou paiement de facture), `customer.subscription.updated/deleted` (mise à jour du statut et de la période).

Modes test et live

Les objets Stripe (produits, prix, webhooks) sont propres à chaque mode. Pour passer en live, recréer produits/prix en mode Live, fournir les clés `sk_live_`/`whsec_live` et créer un webhook Live.

13. E-mails (Resend)

`lib/email/resend.ts` initialise le client (`EMAILS_ENABLED`, `FROM_EMAIL`) ; `lib/email/send.ts` expose les envois (alerte stock, facture, relance, rapport). L'envoi est ignoré proprement si `RESEND_API_KEY` est absente.

Délivrabilité

En mode test (`onboarding@resend.dev`), seuls les e-mails vers l'adresse du compte Resend sont délivrés. Pour la production, vérifier un domaine et définir `RESEND_FROM_EMAIL`.

14. Tâches planifiées (crons)

Deux routes cron, protégées par `CRON_SECRET`, sont déclenchées par Vercel Cron :

- `app/api/cron/daily` : passage des factures échues en « overdue » et envoi des relances (J+7/15/30).
- `app/api/cron/weekly` : envoi du rapport hebdomadaire (lundi).

Les deux respectent le gating des automatisations par plan (`usersWithAutomations`).

```
Authorization: Bearer ${CRON_SECRET}
```

15. Observabilité

- `error_logs + logError()` : capture des erreurs (source, context, message, details).
- `sync_events` : journal des événements de synchro (consultable dans le hub Intégrations).
- `/api/health` : sonde de santé (vérifie la base) renvoyant 200/503.
- Logs runtime Vercel : pour le diagnostic des routes serverless.

16. Rate limiting

`lib/rate-limit.ts` s'appuie sur la fonction `check_rate_limit` (fenêtre fixe, fail-open). Limites par défaut : webhook 600/min, API 120/min, par tenant. Les formulaires publics (démon, contact) sont limités par IP.

17. Déploiement

17.1 Pipeline

Le dépôt GitHub est connecté à Vercel : chaque push sur main déclenche un build et un déploiement de production. Le build exécute la vérification TypeScript (strict).

17.2 Configuration

- Définir toutes les variables d'environnement dans Vercel (Production).
- Un changement de variable nécessite un redéploiement pour être pris en compte.
- Configurer les crons Vercel pour `/api/cron/daily` et `/api/cron/weekly`.

17.3 Migrations base de données

Les évolutions de schéma (tables, fonctions) sont appliquées via des migrations SQL sur le projet Supabase. Conserver l'historique des migrations dans le dépôt.

18. Développement local

18. Cloner le dépôt et installer les dépendances (npm install).
19. Copier les variables dans .env.local (clés Supabase, Stripe test, Resend).
20. Lancer le serveur de développement (npm run dev).
21. Appliquer les migrations sur un projet Supabase de développement.

Astuce typage

Régénérer les types Supabase après un changement de schéma (lib/supabase/database.types.ts) pour conserver un typage strict de la couche données.

19. Sécurité

- RLS active sur toutes les tables métier ; clé service réservée au serveur.
- Clés API hashées (sha256) ; jamais stockées en clair.
- Signature HMAC des webhooks boutiques ; signature Stripe vérifiée.
- Aucune donnée de carte bancaire stockée (déléguée à Stripe).
- Secrets uniquement en variables d'environnement, jamais dans le code.

20. Runbook d'incident

Un webhook boutique n'arrive pas

Vérifier le Journal de synchronisation, la validité de la clé API dans l'URL, l'activation des événements côté boutique, et les logs runtime Vercel sur /api/webhooks/*.

Stripe : « Invalid API Key »

La variable STRIPE_SECRET_KEY est incorrecte (clé publishable au lieu de secrète, mauvais compte/mode, espace parasite). Corriger puis redéployer.

Le webhook Stripe ne met pas à jour l'abonnement

Vérifier que l'endpoint existe dans le bon mode (test/live), que STRIPE_WEBHOOK_SECRET correspond, et l'absence de 400 « signature » dans les logs.

Les e-mails ne partent pas

Vérifier RESEND_API_KEY, l'adresse de notification, et — en prod — un domaine vérifié + RESEND_FROM_EMAIL.

Le build échoue

Erreur TypeScript stricte : lire les logs de build Vercel, corriger le typage, repousser. Possibilité de compiler le projet pour reproduire l'erreur localement.

21. Étendre la plateforme

21.1 Ajouter un forfait

Ajouter une entrée dans PLANS (lib/plans.ts) avec limites, features et priceEnv ; créer le produit/prix Stripe et la variable d'environnement correspondante ; ajouter la carte au module Facturation.

21.2 Ajouter un endpoint API

Créer app/api/v1/<ressource>/route.ts en l'enveloppant dans withApiAuth (auth Bearer + rate limit + gating Business). Filtrer par userId et renvoyer du JSON normalisé.

21.3 Ajouter une automatisation

Ajouter une colonne booléenne à user_settings, l'exposer dans le module Automatisations (gaté par hasAutomations) et brancher son exécution au bon endroit (core sync ou cron).

22. Annexe — Référence des endpoints

Endpoint	Méthode	Rôle
/api/v1/products	GET, POST	Lister / créer des produits (Business).
/api/v1/orders	GET, POST	Lister / créer des commandes (Business).
/api/v1/customers	GET	Lister les clients (Business).
/api/v1/forecast	GET	Prévisions de stock (Business).
/api/webhooks/woocommerce	POST	Webhooks WooCommerce.
/api/webhooks/shopify	POST	Webhooks Shopify.
/api/webhooks/stripe	POST	Webhooks Stripe (abonnements).
/api/billing/checkout	POST	Session Checkout abonnement.
/api/export/[entity]	GET	Exports CSV (products, orders, customers, invoices, report-*).
/api/cron/daily	GET	Cron quotidien (relances).
/api/cron/weekly	GET	Cron hebdomadaire (rapport).
/api/health	GET	Sonde de santé.

La documentation API détaillée (authentification, schémas, exemples) fait l'objet d'un document dédié.